TinyML Compression and Feature Engineering for Real-Time IMU Gesture Recognition on Arduino Nano 33 BLE

Mohammadreza Zamani
Erasmus Student
Università di Trento
Trento, Italy
mohammadreza.zamani@studenti.unitn.it

Asal Abbas Nejad Fard
Erasmus Student
Università di Trento
Trento, Italy
asal.abbasnejadfard@studenti.unitn.it

ABSTRACT

This paper investigates efficient TinyML solutions for real-time inertial measurement unit (IMU)-based gesture recognition on a resource-constrained microcontroller.

Two optimization strategies are evaluated: neural-network compression and feature engineering. Standard and depthwise CNNs are compared in FP32 and INT8 formats, while raw, RMS, FFT, and spectral representations are evaluated using a controlled fully-connected classifier.

The final RMS Tiny INT8 model achieves 100% accuracy on the fixed 32-window test set while requiring only 284 parameters, 256 neural-network MAC operations, and a 3.414 KB TensorFlow Lite model. Deployed on an Arduino Nano 33 BLE using TensorFlow Lite Micro, the model requires 0.165 ms for neural-network inference and 0.346 ms for the complete processing pipeline.

KEYWORDS

TinyML, IMU, Gesture Recognition, TensorFlow Lite Micro, INT8 Quantization, Arduino Nano 33 BLE

1 INTRODUCTION

Tiny Machine Learning (TinyML) enables machine-learning inference directly on embedded devices with limited memory, energy, and computational resources. Recent work has demonstrated the potential of on-device time-series classification and human activity recognition on resource-constrained platforms [2, 5].

Deploying neural networks on microcontrollers requires balancing classification performance against model complexity, memory footprint, and inference latency. Physical hardware measurements are therefore important when evaluating practical embedded AI systems [1, 3].

This work investigates two complementary strategies for efficient IMU gesture recognition. The first evaluates neural-network compression through lightweight CNN architectures and INT8 quantization. The second evaluates whether low-dimensional handcrafted representations can replace direct processing of the raw IMU window.

Table 1: Gesture Classes

Class ID	Gesture
0	circle
1	left_right
2	rest
3	up_down

The main contributions are:

- Comparison of Standard and Depthwise CNN architectures in FP32 and INT8 formats.
- Controlled comparison of Raw, RMS, FFT, and Spectral input representations.
- Development of a highly compact RMS Tiny INT8 model.
- Physical deployment and latency/memory benchmarking on an Arduino Nano 33 BLE.

The complete source code, datasets, deployment files, and experimental results are publicly available at: https://github.com/mrzamaniiii/TinyML-Compression-Benchmark.

2 SYSTEM AND EXPERIMENTAL SETUP

The system processes six-axis IMU data consisting of three accelerometer channels (aX, aY, aZ) and three gyroscope channels (gX, gY, gZ).

The processing pipeline is:

6-Axis IMU → 128-Sample Window → Preprocessing / Feature Extraction
→ INT8 TinyML Model → Gesture Prediction

Each input window contains

$$128 \times 6 = 768$$

raw sensor values.

The dataset contains 320 windows distributed equally across four gesture classes, with 80 windows per class.

A fixed stratified 80/10/10 split is used for the main experiments:

Table 2: Study 1 Offline Model Comparison

Model	Params.	NN MACs	Size (KB)	Acc.
Standard CNN FP32	2660	160320	15.121	100%
Standard CNN INT8	2660	160320	9.797	100%
Depthwise CNN FP32	1330	52544	10.824	100%
Depthwise CNN INT8	1330	52544	10.125	100%

- Training: 256 windows
- Validation: 32 windows
- Test: 32 windows

The test set therefore contains eight windows from each gesture class. All normalization statistics are estimated using only the training set and then applied unchanged to the validation and test sets, preventing evaluation information from entering the training pipeline. The final embedded implementations use TensorFlow Lite Micro and INT8 inference directly on the Arduino Nano 33 BLE without cloud processing.

3 STUDY 1: NEURAL NETWORK COMPRESSION

The first study investigates the effect of neural-network architecture and INT8 post-training quantization on raw-signal gesture classification.

Two convolutional architectures are evaluated: a Standard CNN and a Depthwise-separable CNN. Depthwise-separable convolutions are commonly used to reduce computational complexity in resource-constrained sensor-classification systems [4].

Both models operate directly on the normalized 128×6 IMU window and use identical training, validation, and test partitions.

Replacing standard convolutions with depthwise-separable convolutions reduces the parameter count from 2,660 to 1,330 and the neural-network MAC count from 160,320 to 52,544. This corresponds to reductions of 50% and approximately 67.2%, respectively. INT8 quantization reduces the Standard CNN TFLite model size from 15.121 KB to 9.797 KB. The reduction is smaller for the Depthwise CNN because its parameter set is already compact and TensorFlow Lite metadata represents a larger fraction of the final model file.

Although the Depthwise CNN requires fewer parameters and MAC operations, its measured Flash and Tensor Arena requirements are slightly higher. Nevertheless, it reduces measured inference latency from 25.370 ms to 16.880 ms, corresponding to approximately a 33.5% reduction.

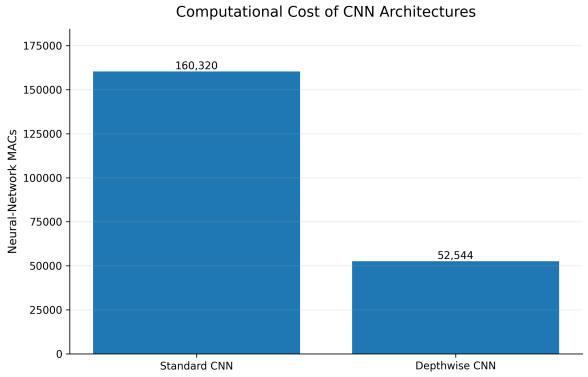

Figure 1: Neural-network MAC comparison of the Standard and Depthwise CNN architectures.

Table 3: Study 1 Arduino INT8 Deployment Results

Metric	Standard CNN	Depthwise CNN
Flash (B)	194408	205424
Tensor Arena (B)	6644	7156
Inference (ms)	25.370	16.880
Total (ms)	26.760	18.330

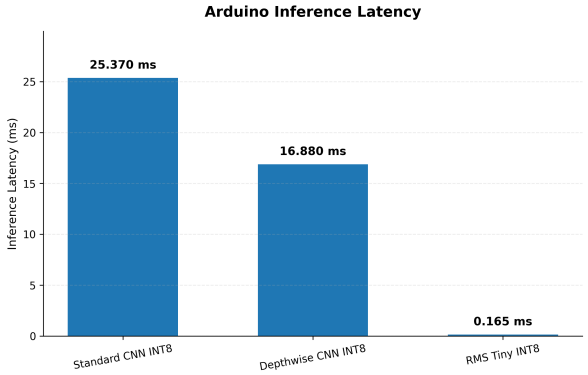

Figure 2: Measured inference latency on the Arduino Nano 33 BLE for the three final INT8 models.

4 STUDY 2: FEATURE ENGINEERING

The second study investigates whether feature engineering can provide a more efficient representation than directly processing the raw IMU window.

Four input representations are evaluated: Raw Signal, Root Mean Square (RMS), Fast Fourier Transform (FFT), and Spectral features.

To isolate the effect of the representation, all four experiments use the same classifier structure:

$$\text{Input} \longrightarrow \text{Dense}(64) \longrightarrow \text{Dense}(32) \longrightarrow \text{Dense}(4)$$

Table 4: Feature Representation Comparison

Representation	Features	Params.	Acc.	Macro F1
Raw Signal	768	51428	100.000%	1.0000
RMS	6	2660	100.000%	1.0000
FFT	48	5348	100.000%	1.0000
Spectral	12	3044	96.875%	0.9686

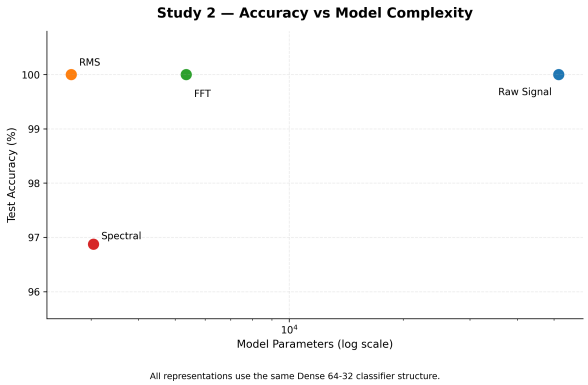

Figure 3: Test accuracy versus parameter count for the controlled feature representations.

Only the input dimensionality and representation change between experiments.

The raw representation uses all 768 values from each IMU window. RMS reduces the same window to only six features, one for each IMU channel. Despite this dimensionality reduction, RMS achieves the same test accuracy and Macro F1 as both Raw Signal and FFT. The Spectral representation uses 12 features but achieves 96.875% accuracy, indicating that additional compression removes some information useful for classification.

The controlled comparison therefore identifies RMS as the most efficient representation among those achieving 100% test accuracy: it reduces the input dimension from 768 raw values to six features and reduces the classifier size from 51,428 to 2,660 parameters.

5 FINAL RMS TINY INT8 MODEL

Following the controlled feature-engineering study, RMS was selected for the final lightweight embedded implementation.

For each IMU channel, the RMS feature is computed over the 128-sample window as

$$\mathrm{RMS}_c = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x_c[n]^2}, \tag{1}$$

where $N = 128$ and c denotes one of the six IMU channels.

Table 5: Final RMS Tiny Model Characteristics

Metric	Value
Input features	6
Parameters	284
NN MACs	256
FP32 TFLite size	3.227 KB
INT8 TFLite size	3.414 KB
INT8 test accuracy	100.000%
INT8 Macro F1	1.0000

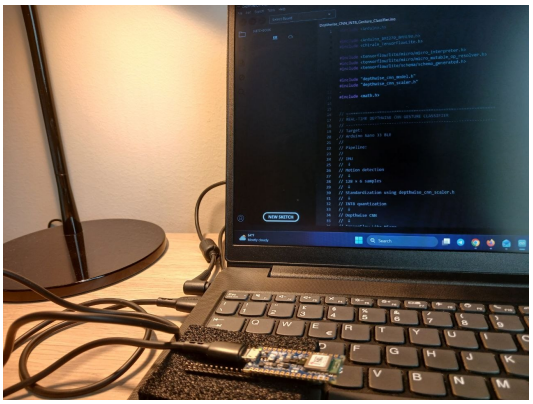

Figure 4: Real hardware setup used for TinyML deployment and embedded benchmarking on the Arduino Nano 33 BLE.

The resulting six RMS values are standardized using mean and scale parameters estimated exclusively from the training set and passed to the optimized classifier:

6 RMS Features → Dense(16) → Dense(8) → Softmax(4)

The model contains only 284 trainable parameters and requires 256 neural-network MAC operations per inference.

Interestingly, its INT8 TFLite file is slightly larger than the FP32 file. For such a small neural network, quantization metadata represents a non-negligible fraction of the final file size. Therefore, INT8 is used mainly to enable integer microcontroller inference rather than additional file-size reduction.

The reported 256 MAC operations refer only to the neural-network classifier and exclude the RMS feature-extraction cost.

6 EMBEDDED DEPLOYMENT RESULTS

The three final INT8 models were deployed and evaluated on a physical Arduino Nano 33 BLE using TensorFlow Lite Micro.

Table 6: Final Arduino Deployment Comparison

Model	Flash (B)	Arena (B)	Inf. (ms)	Total (ms)
Standard INT8	194408	6644	25.370	26.760
Depthwise INT8	205424	7156	16.880	18.330
RMS Tiny INT8	165960	948	0.165	0.346

Figure 4 shows the real experimental setup used for physical deployment and on-device measurements.

For Standard and Depthwise CNNs, preprocessing consists primarily of raw-signal normalization. RMS Tiny additionally performs RMS feature extraction and feature standardization.

RMS Tiny requires 0.181 ms for RMS extraction and preprocessing, 0.165 ms for neural-network inference, and 0.346 ms for the complete processing pipeline.

Compared with Standard CNN INT8, RMS Tiny reduces neural-network inference latency by approximately 99.35% and total processing time by approximately 98.71%.

Compared with Depthwise CNN INT8, the corresponding reductions are approximately 99.02% and 98.11%.

The Tensor Arena requirement falls from 6,644 bytes for Standard CNN and 7,156 bytes for Depthwise CNN to only 948 bytes for RMS Tiny.

Measured Flash usage is 165,960 bytes, approximately 14.6% lower than Standard CNN and 19.2% lower than Depthwise CNN.

These measurements reinforce the importance of physical latency and resource evaluation for embedded AI systems [1, 3].

7 ROBUSTNESS CHECK

Additional checks were performed to investigate whether the strong test performance could be explained by direct duplicate leakage or only by the primary random split.

No exact duplicate windows were found across the dataset partitions.

An additional blocked 60/20/20 split was also evaluated. Under this split, the raw-signal 1-nearest-neighbor baseline and the RMS Tiny FP32 classifier both achieved 100% accuracy on the corresponding 64-window test subset.

These results provide an additional consistency check, although the blocked experiment should be interpreted descriptively because it also reduces the training fraction from 80% to 60% and therefore does not isolate temporal blocking as the only experimental change.

8 DISCUSSION

The experiments show that TinyML efficiency can be improved through both neural-network compression and feature engineering.

Depthwise CNN reduces the neural-network MAC count substantially and also improves measured inference latency, although its Flash and Tensor Arena requirements are slightly higher than those of the Standard CNN.

Feature engineering provides the largest overall reduction in this study. RMS compresses each 128×6 window from 768 raw sensor values to six features while maintaining 100% accuracy on the current test set. This enables the final RMS Tiny INT8 classifier to use only 284 parameters and 256 neural-network MACs.

The reported 100% accuracy should nevertheless be interpreted in the context of the current 32-window main test set. Larger datasets, multiple users, and independent recording sessions are required for a broader evaluation of real-world generalization.

Power and energy consumption were not directly measured in this work. The embedded comparison therefore focuses on measured execution latency, Flash usage, Tensor Arena requirements, model size, and computational complexity.

9 CONCLUSION

This work evaluated neural-network compression and feature engineering for TinyML-based IMU gesture recognition.

The Depthwise CNN reduced computational complexity and inference latency relative to the Standard CNN, while RMS feature engineering provided the largest overall efficiency improvement.

The final RMS Tiny INT8 model achieved 100% accuracy on the current test set using only 284 parameters and 256 neural-network MACs. On the Arduino Nano 33 BLE, it required 0.165 ms for inference and 0.346 ms for the complete processing pipeline.

These results demonstrate that lightweight signal processing combined with compact neural networks is a promising approach for efficient TinyML deployment on resource-constrained devices.

ACKNOWLEDGEMENTS

The authors thank Prof. Kasim Sinan Yildirim for his guidance and supervision throughout this project.

REFERENCES

[1] Leonardo Lucio Custode, Pietro Farina, Eren Yildiz, Renan Beran Kilic, Kasim Sinan Yildirim, and Giovanni Iacca. 2024. Fast-Inf: Ultra-Fast Embedded Intelligence on the Batteryless Edge. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems (SenSys '24)*. ACM, 239–252. https://doi.org/10.1145/3666025.3699335

[2] Imola Fodor, Roberto Lorenzon, Gilberto Pin, Stefano Antonello, and Kasim Sinan Yildirim. 2025. Embedded-AI Based Fault Detection in Domestic Dishwashers: A Tale of On-Device Time Series Deep Classification. *IFAC-PapersOnLine* 59, 26 (2025), 371–376. https://doi.org/10.1016/j.ifacol.2025.12.063

[3] Mohammad Amin Hasanpour, Mikkel Kirkegaard, and Xenofon Fafoutis. 2025. EdgeMark: An Automation and Benchmarking System for Embedded Artificial Intelligence Tools. *Journal of Systems Architecture* 167 (2025), 103488. https://doi.org/10.1016/j.sysarc.2025.103488

[4] Imran Hossan, Mst. Nusratul Jannat Mary, and Mohammod Abdul Motin. 2025. TinyHAR-Net: Design and Implementation of a TinyML-Based Human Activity Recognition Framework on STM32. *IEEE Embedded Systems Letters* (2025). https://doi.org/10.1109/LES.2025.3639944

[5] Haotian Zhou, Xiujun Zhang, Yu Feng, Tongda Zhang, and Lijuan Xiong. 2025. Efficient Human Activity Recognition on Edge Devices Using DeepConv LSTM Architectures. *Scientific Reports* 15 (2025), 13830. https://doi.org/10.1038/s41598-025-98571-2